Actividad 3.3 - Ajuste fino de LLM

Fine Tuning

Adrián García García

Modelos de Inteligencia Artificial

ÍNDICE

1. Planteamiento de la tarea y necesidades
2. Selección del Modelo
3. Pruebas de Comportamiento Base (Baseline)
4. Gestión del Conjunto de Entrenamiento
5. Proceso de Fine-Tuning
6. Evaluación Comparativa y Reflexión
7. Publicación del modelo
8. Publicación de espacios
9. Enlaces a repositorios de Hugging Face

Planteamiento de la tarea y necesidades

Sony está realizando la tercera temporada de The Last of Us, para ello necesita actores/actrices, de forma que hacen un casting para elegir a los mejores. Una vez pasado el casting, los actores quieren saber acerca del videojuego para conocer exactamente su papel y meterse más en él, pero no les gusta jugar a videojuegos. Para ello, Sony ha tenido la idea de tener un modelo que sepa sobre la saga The Last of Us de forma más detallada, así los actores pueden preguntarle cosas al modelo para aprender más sobre la saga.

En este caso, se ha llegado a un acuerdo para hacer Fine-Tuning a un modelo para adaptarlo acerca de la saga en cuestión.

Selección del Modelo

El modelo para hacer Fine-Tuning será Qwen/Qwen2.5-7B-Instruct, perteneciente a la familia de modelos Qwen, diseñados para tareas de lenguaje natural, razonamiento, programación y conversación. Se ha elegido este modelo debido a que el dataset que se va a utilizar presenta un formato tipo chat (con roles ‘system’, ‘user’ y ‘assistant’), ya que los modelos ‘Instruct’ funcionan especialmente bien con este tipo de estructura, ofreciendo un mejor rendimiento durante el proceso de fine-tuning. Además, este modelo se ha elegido frente a alternativas como LLaMA o Mistral, dado que ofrece un buen equilibrio entre rendimiento y coste computacional, junto con un sólido soporte multilingüe y una licencia permisiva como Apache License 2.0, lo que facilita su uso, modificación y distribución en entornos académicos y profesionales.

En cuanto al dataset con el que fue pre-entrenado, para esta versión de Qwen, se ha pasado de un dataset de 3 trillones de tokens (con los que se entrenaba en versiones anteriores) a 7 trillones de tokens, soportando aproximadamente unos 30 idiomas, entre ellos se encuentra el español y el inglés.

El corpus de entrenamiento contiene diferentes tipos de datos:

- Texto web filtrado
- Documentación técnica
- Código fuente
- Datos matemáticos
- Contenido multilingüe
- Conocimiento general

Además del pre-training, también hay una época post-training, donde se le pasa un conjunto de datos (según la documentación aproximadamente 500000 datos) acerca de programación, razonamiento, matemáticas… y se hace una evaluación de ello.

Gracias a este tipo de entrenamiento, el modelo base es capaz de comprender el lenguaje natural, generar texto, programar, resolver problemas matemáticos y razonar.

Por último, el número de parámetros exactos que tiene este modelo es de 7.61 billones de parámetros, cuya arquitectura está basada en los transformers (mecanismo de atención).

En conjunto, estas características hacen que el modelo seleccionado sea adecuado para la tarea planteada, con un buen rendimiento, eficiencia y facilidad de uso.

Pruebas de Comportamiento Base (Baseline)

A continuación, se realizará una serie de pruebas con el modelo base para evaluar su comportamiento en la tarea sobre la que posteriormente se llevará a cabo el fine-tuning. La metodología seguida será la siguiente:

- Creamos el pipeline con el modelo correspondiente
- Se crea un diccionario con quince preguntas, algunas son básicas que el modelo quizá pueda responder correctamente, y otras son preguntas más concretas acerca del videojuego/serie
- Recorremos las diferentes preguntas y se lo pasamos al modelo base
- Estas han sido las respuestas del modelo a las preguntas anteriores:
    - Pregunta 1
        - pregunta: What does Ellie find out when she returns to her house on The Last of Us?
        - respuesta: When Ellie returns to her house in The Last of Us, she finds it ransacked and her family's remains inside, which deeply affects her emotionally and drives her further into the story's conflict.
    - Pregunta 2
        - pregunta: Why does Tommy go to Santa Barbara ahead of Ellie in The Last of Us?
        - respuesta: In The Last of Us, Tommy goes to Santa Barbara ahead of Ellie because he is trying to find a safe place for her and other survivors. Specifically, Tommy has information about a

secure location in Santa Barbara that might offer them protection from the infected and other dangers they face. He travels there first to ensure the area is safe before bringing Ellie and the others.

- Pregunta 3
  - pregunta: What happened in the event "Ellie goes to Seattle" in The Last of Us?
  - respuesta: In "The Last of Us," during the event "Ellie goes to Seattle," Ellie travels to Seattle with Joel to deliver a package. However, they become separated when Ellie is taken by a group of survivors who believe she has valuable information about a cure for the fungal infection that has caused the zombie outbreak. Joel, believing Ellie dead, returns to Philadelphia. This event is significant as it sets up Ellie's journey to find Joel and her own path in the post-apocalyptic world.
- Pregunta 4
  - pregunta: Why does Joel kill Marlene in The Last of Us?
  - respuesta: In The Last of Us, Joel kills Marlene because she poses a significant threat to Ellie and his mission. Marlene discovers that Joel is hiding Ellie and attempts to take her by force. When Joel tries to reason with her, Marlene becomes violent and attacks Joel, putting Ellie in danger. To protect Ellie, Joel is forced to defend himself and ultimately kills Marlene. This action is a pivotal moment in the story, highlighting the harsh realities and moral dilemmas faced by survivors in the game's post-apocalyptic world.
- Pregunta 5
  - pregunta: How does the rat king differ from other infected in The Last of Us?
  - respuesta: In The Last of Us, the Rat King is a highly mutated and extremely aggressive variant of the infected rats that have been altered by the fungal infection. It differs from other infected in several key ways:
    - Size: The Rat King is significantly larger than normal infected rats, often described as the size of a small dog.

- Strength and Durability: It possesses enhanced strength and durability, making it much harder to defeat compared to regular infected rats.
- Aggression: The Rat King exhibits an even more frenzied and relentless aggression towards humans, driven by the infection's influence.
- Fungal Mutation: It has undergone more extensive fungal mutation, which likely gives it unique abilities or traits not seen in other infected animals.
- Role in the Story: The Rat King plays a significant role in the narrative, serving as a formidable obstacle for the main characters and symbolizing the pervasive threat of the infection.

- Pregunta 6
  - pregunta: How do stalkers compare to runners in terms of strength in The Last of Us?
  - respuesta: In The Last of Us, stalkers and runners are both types of infected enemies, but they differ significantly in strength and behavior:

    Runners:

    - Are slower and less aggressive.
    - Have lower health and are easier to kill.
    - Typically appear in groups and can be dispatched with fewer resources.

    Stalkers:

    - Move much faster and are more aggressive.
    - Have higher health compared to runners.
    - Are harder to hit due to their speed and agility.
    - Often appear alone or in smaller numbers, making them more challenging to take down.

    In summary, stalkers are generally stronger and more difficult to defeat compared to runners.

- Pregunta 7

        - pregunta: What is the main reason for Ellie's journey to Seattle in The Last of Us?
        - respuesta: In The Last of Us, Ellie travels to Seattle to find her father, who was last known to be in the area. Her journey is driven by a desire to reconnect with her father and learn more about her past, as well as to seek safety in a rumored safe haven.
    - Pregunta 8
        - pregunta: What happens to Sarah in Texas on The Last of Us?
        - respuesta: In "The Last of Us," Sarah is initially introduced as a survivor who takes Joel and Ellie under her protection after they meet in Nebraska. When the trio reaches Texas, Sarah becomes more involved with Ellie, showing interest and concern for her well-being. However, Sarah's character does not survive the Texas portion of the game. She is killed by an infected zombie near the beginning of their journey through the state, which affects Joel and Ellie deeply. Her death is a significant moment in the story, highlighting the harsh realities of survival in a post-apocalyptic world.
    - Pregunta 9
        - pregunta: What happens to Sam after he is scratched by an infected in The Last of Us?
        - respuesta: After being scratched by an infected in The Last of Us, Sam is infected himself. His condition progressively worsens over time, leading to him becoming aggressive and dangerous. Joel ultimately has to make the difficult decision to put Sam down to prevent further harm to themselves and others.
    - Pregunta 10:
        - pregunta: What happened to Ellie and Joel after they left the University on The Last of Us?
        - respuesta: After leaving the University, Ellie and Joel's journey becomes more perilous as they face increasing threats from infected and other survivors. They eventually part ways due to their different survival philosophies and Joel's deteriorating condition. Joel is later killed by an infected, and Ellie continues her journey alone, ultimately reaching Feros with Abby and meeting up with other survivors.

- Pregunta 11:
  - pregunta: What is the relationship between Jerry and Abby in The Last of Us?
  - respuesta: In The Last of Us, Jerry and Abby are brother and sister. Jerry is a key character who helps Joel and Ellie early in their journey, and Abby is his young daughter
- Pregunta 12:
  - pregunta: How did Abby react to Owen's death in The Last of Us?
  - respuesta: In The Last of Us, Abby's reaction to Owen's death is deeply emotional and intense. She becomes extremely distraught, showing signs of grief and anger. Her relationship with Owen, who was like a father figure to her, makes her reaction particularly raw and personal. Abby is seen crying and expressing her sorrow, reflecting the strong bond she had with him
- Pregunta 13:
  - pregunta: What motivated Abby to spare Ellie's life in The Last of Us?
  - respuesta: In The Last of Us, Abby spares Ellie's life primarily due to her own past experiences with abandonment and loss. Abby, who was taken from her home as a child, empathizes with Ellie's situation and recognizes the value of survival. This connection and shared trauma motivate Abby to protect Ellie, seeing in her a younger version of herself. Additionally, Ellie's intelligence and resourcefulness make her a valuable ally in their journey
- Pregunta 14:
  - pregunta: Why do Clickers develop a form of echolocation in The Last of Us?
  - respuesta: In The Last of Us, Clickers develop echolocation as a survival mechanism due to their blindness caused by the fungal infection. This adaptation allows them to navigate and hunt effectively in their dark, post-apocalyptic environment. Echolocation helps them detect and locate prey or threats through sound waves, enabling them to compensate for their visual impairment and remain formidable survivors

- Pregunta 15:
  - pregunta: What are the four categories of collectibles in The Last of Us?
  - respuesta: In The Last of Us, there are four main categories of collectibles:
    1. Blueprints: These provide insight into crafting recipes and can be found in various locations.
    2. Notes: These offer backstory and context to the game's world and characters.
    3. Audio Logs: These are recordings that provide additional narrative depth and character development.
    4. Photographs: These are historical images that add to the game's lore and setting.

De todas las respuestas que nos ha dado, se puede decir que el modelo base hace muchas alucinaciones y solo ha respondido bien a tres preguntas (dos de ellas preguntas bastante básicas), a la **pregunta 5** acerca del Rat King, la **pregunta 6** acerca de la diferencia entre Runners y Stalkers, la **pregunta 9** acerca de lo ocurrido con Sam después de que le mordieran, la **pregunta 12** sobre la reacción de Abby a la muerte de Owen y la **pregunta 14** acerca del porqué los Clickers habían desarrollado la habilidad de ecolocalización

A continuación se va a justificar porque las otras preguntas son erróneas. En la **primera pregunta** (What does Ellie find out when she returns to her house on The Last of Us?) el modelo base dice que encuentra su casa saqueada y con los restos de su familia en su interior, sumergiendo a Ellie aún más en el conflicto de la historia y eso no es verdad, cuando Ellie vuelve a su casa al final del juego, descubre que su pareja se ha marchado junto con su hijo, dando así el cierre de la historia de la segunda parte.

Por otro lado, en la **segunda pregunta** (Why does Tommy go to Santa Barbara ahead of Ellie in The Last of Us?) el modelo responde diciendo que Tommy fue antes que Ellie a Santa Bárbara para encontrar un lugar seguro para ella y otros supervivientes. De nuevo, el modelo se confunde aquí, ya que lo que ocurrió en verdad es que Tommy fue a Santa Bárbara para buscar a Abby y poder terminar la venganza y no para buscar un lugar seguro.

En cuanto a la **tercera pregunta** (What happened in the event "Ellie goes to Seattle" in The Last of Us?), el modelo dice que Ellie va a Seattle con Joel para llevar un paquete, aqui el modelo claramente está alucinando, precisamente Ellie va a Seattle a buscar a los responsables de la muerte de Joel (minuto 2:38).

Por otra parte, la respuesta de la **cuarta pregunta** va bien encaminado, pero veo que el modelo alucina un poco a la hora de responder, ya que dice que Joel intenta

razonar con Marlene y ella le ataca, cuando realmente pasó lo contrario, Marlene está dispuesta a soltar el arma para poder hablar, pero es Joel quien ataca primero.

En cuanto a la **séptima pregunta** (What is the main reason for Ellie's journey to Seattle in The Last of Us?), el modelo responde diciendo que Ellie viaja a Seattle para buscar a su padre, pero como vimos antes, Ellie viaja a Seattle por venganza.

Por otro lado, en la **octava pregunta** (What happens to Sarah in Texas on The Last of Us?), el modelo se vuelve a equivocar diciendo que Sarah se presenta como una superviviente que acoge a Joel y a Ellie, cuando en realidad Sarah (hija de Joel) muere al principio del juego a manos de un soldado.

Luego tenemos la **décima pregunta** (What happened to Ellie and Joel after they left the University on The Last of Us?), el modelo cuenta que los caminos se separan debido a la diferencia de filosofía de supervivencia, y no es así, lo que ocurre realmente es que Joel está en un estado crítico y Ellie tiene que apañárselas para poder curar a Joel

Después, la **undécima pregunta** (What is the relationship between Jerry and Abby in The Last of Us?), el modelo responde diciendo que eran hermanos, cuando realmente Jerry es el padre de Abby

Por otro lado, en la **decimotercera pregunta** (What motivated Abby to spare Ellie's life in The Last of Us?) el modelo hace una alucinación muy grande diciendo que Abby empatiza con Ellie debido a experiencias de abandono del pasado, pero lo que ocurre de verdad es que Abby perdona la vida a Ellie debido a que Dina está embarazada y Lev le ruega que no la mate

Por último, en la **decimoquinta pregunta** (What are the four categories of collectibles in The Last of Us?) el modelo iba bien encaminado pero no es la respuesta correcta, los coleccionables de The Last of Us son artefactos, manuales de entrenamiento, cartas y monedas/pendientes de luciérnagas

Podemos ver que el modelo base comete muchos errores, adivinando solo un tercio de las preguntas. También comete muchas alucinaciones inventando situaciones que nunca pasaron. Ahora vamos a proceder a gestionar el dataset para poder empezar el Fine-Tuning.

Gestión del Conjunto de Entrenamiento

Para este caso, no he encontrado ningún dataset que se adecúe al problema, por lo que he decidido crearlo por mi cuenta. Para ello he seguido la siguiente estrategia, he instalado ollama para descargarme un modelo local, en este caso 'llama3:instruct' para descargar dicho modelo, necesitamos ejecutar el comando 'ollama pull llama3:instruct'.

Una vez instalado, creamos un script que haga un scrapping a la API del fandom de The Last of Us por diferentes categorías (personajes, facciones, localizaciones…) para obtener información acerca del universo de The Last of Us, de esta forma, el modelo aprende para solucionar el problema que se comentó al principio (actores que quieren saber acerca de la saga). Una vez hecho scrapping, se divide en "chunks" para pasarselo poco a poco al modelo local y éste se encarga de generar un JSON con la información del texto o "chunk" pasado. Una vez nos lo genera, lo tratamos para pasarlo a un formato chat (system, user y assistant), que es lo recomendable para hacer Fine-Tuning y lo guardamos en un JSON (En este enlace se adjunta el script python donde se ha hecho todo lo explicado), lo recomendable es ir haciéndolo en conjuntos de categorías y no todo junto, ya que al ser un modelo local y, en mi caso, tener un portátil, tarda mucho tiempo.

El tiempo total tardado en hacer todos los JSON y unificarlos son los siguientes:

- Primeras pruebas con pequeñas categorías, donde ha habido errores y mejoras: 5-6 horas
- Una vez hechas las pruebas, generamos los distintos JSON que utilizaremos. Para la categoría de la historia (6 categorías), ha tardado un total de 4 horas
- Para la categoría llamada 'Gameplay' (los diferentes objetos, objetos que se pueden crear…) también unas 4 horas
- Para la categoría de las facciones ha tardado bastante más, unas 7-8 horas
- Para la categoría llamada 'Mundos', ha tardado una hora y media
- Por último, para la categoría llamada 'Narrativa' ha tardado unas 3-4 horas

Haciendo el recuento, ha tardado aproximadamente unas 27 horas y media en generar el dataset

Una vez generado el dataset, se procederá a subirlo a una cuenta de Hugging Face. Para ello, se utilizará código, ya que se considera la forma más sencilla de hacerlo. El código es el siguiente:

```
from datasets import load_dataset
from huggingface_hub import login

login()
dataset = load_dataset("json", data_files="./tlou_dataset.jsonl")

dataset = dataset["train"].train_test_split(test_size=0.1)

dataset.push_to_hub("adriangg04/the-last-of-us-instruction-dataset")
```

Vemos que carga el JSONL de nuestro equipo, divide en datos de train y test y se sube al repositorio creado del dataset

Una vez subido, lo cargamos en el código para poder hacer Fine-Tuning con dicho dataset, hacemos uso del método “load_dataset()” indicando el nombre del repositorio del dataset (‘adriangg04/the-last-of-us-instruction-dataset’). Como el modelo se ha subido con train y test incluidos, no hace falta hacer split. Una vez cargado el dataset, le aplicamos una función llamada “format_chat” tanto al train como al split, esta función se encarga de transformar cada ejemplo del dataset en un formato de texto plano que el modelo puede entender para entrenamiento supervisado. De forma más detallada, toma la lista de mensajes de usuario, asistente y sistema, los organiza siguiendo el template del tokenizer del modelo, y devuelve un string con los roles indicados (user, assistant y system). Esto permite que el modelo aprenda a generar respuestas coherentes en conversaciones, simplificando la estructura compleja del JSON original a un único campo "text" listo para tokenizar y usar en el SFTTrainer

```
def format_chat(example):
    text = tokenizer.apply_chat_template(
        example["messages"],
        tokenize=False,
        add_generation_prompt=False
    )
    return {"text": text}
```

Proceso de Fine-Tuning

Vamos a pasar ahora a explicar los distintos hiperparámetros utilizados.

En primer lugar tenemos la clase ‘BitsAndBytesConfig’ que nos va a permitir cargar el modelo cuantizado en 4 bits, de forma que podremos hacer QLoRa, para ello tenemos que dar valor a los siguientes hiperparámetros:

- ‘load_in_4bit=True’: Nos permitirá precisamente cargar el modelo en cuantización de 4 bits, de forma que consume mucho menos VRAM
- ‘bnb_4bit_quant_type="nf4"’: Define el tipo de cuantización de 4 bits, en este caso se usa ‘nf4’ ya que es el más óptimo para QLoRa
- ‘bnb_4bit_compute_dtype=torch.bfloat16’: Aunque el modelo está cuantizado en 4 bits, a la hora de hacer los cálculos se realizan utilizando el formato

bfloat16 (se descuantiza temporalmente), de esta manera no se pierde tanta precisión durante el entrenamiento

- ‘bnb_4bit_use_double_quant=True’: Cuando se cuantiza un modelo, se necesita almacenar unas constantes de escalado, pues con este parámetro a True, nos permite cuantizar también dichas constantes, eso nos permite ahorrar memoria VRAM y poder hacer Fine-Tuning de modelos grandes en GPU’s no tan potentes

Esta clase la pasamos al cargar el modelo base al que le vamos a hacer Fine-Tuning, de esta manera, se cuantiza en 4 bits

Ahora vamos a pasar con los hiperparámetros de la clase LoraConfig, que nos va a permitir configurar el entrenamiento LoRa:

- ‘r=16’: Lo que representa el hiperparámetro ‘r’ es la dimensión de las matrices de bajo rango, ya que LoRA no modifica directamente los pesos originales del modelo. En lugar de eso, añade una actualización de bajo rango y aprende dos matrices más pequeñas, pues esta ‘r’ es el tamaño de estas matrices. De esta forma, el valor de r controla la capacidad de adaptación, es decir, la cantidad de información nueva que el modelo puede aprender durante el fine-tuning. En este caso he establecido un valor de 16, debido a que hay un buen equilibrio entre la capacidad que tiene el modelo de aprender durante el entrenamiento y el coste computacional, asegurando que el modelo aprenda sobre el dataset que le pasamos para entrenar. Además, un r muy grande requiere de mucho más recursos de lo que se dispone

- ‘lore_alpha=32’: Este parámetro determina cuánto peso tienen las actualizaciones de LoRA sobre los pesos originales del modelo base. Normalmente se suele poner el doble del valor establecido en ‘r’, por lo que en este caso tendrá un valor de 32

- ‘target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]’: Este parámetro indica que capas queremos adaptar, en este caso se ha elegido la capa de atención y las capas del perceptrón multicapa. Se incluyen ambas capas ya que así el fine-tuning tiene más capacidad de adaptación sin entrenar todo el modelo, aunque esto conlleva entrenar más números de parámetros

- ‘lora_dropout=0.05’: Este parámetro permite aplicar regularización durante el entrenamiento, reduciendo el riesgo de que el modelo se sobreajuste al conjunto de datos de fine-tuning

- ‘bias=”none”’: En este parámetro se indica si se adaptan los ‘bias’ del modelo base, en este caso no es tan necesario, ya que no son parámetros tan importantes para adaptar el modelo. Por lo que lo indicamos a “none” para no

entrenar dichos parámetros, de esta forma aumenta la eficiencia y se minimiza el número de parámetros a entrenar

- ‘task_type=”CASUAL_LM”’: Esto indica el tipo de modelo al que le vamos a hacer Fine-Tuning, en este caso, como Qwen es un modelo de tipo ‘CASUAL_LM’, indicamos dicho valor

Por otro lado, vamos a pasar a la clase que tiene más parámetros ‘TrainingArguments’, vamos a explicar las más importantes:

- ‘per_device_batch_size=4’: Este hiperparámetro indica el número de ejemplos procesados simultáneamente por cada GPU en un paso de entrenamiento, es decir, la cantidad de ejemplos procesados en paralelo en un solo paso. En este caso, como estamos haciendo el entrenamiento en un entorno en la nube, le ponemos el valor de 4, que para una gráfica T4 está bastante bien, consumiendo aproximadamente 14 GB de 15 GB de la GPU

- ‘gradient_accumulation_steps=4’: Esto permite simular un batch más grande antes de actualizar los pesos sin necesidad de una GPU con muchísima memoria. Buscando más información, este parámetro se ‘combina’ con el anterior, de manera que se multiplica este valor con el anterior (4x4) dando un valor de 16. Esto significa que el modelo actualiza sus pesos después de procesar 16 ejemplos, aunque solo se carguen 4 en memoria en cada paso de entrenamiento. Por lo que un valor de 4 es bastante equilibrado, permitiendo que el modelo aprenda durante el entrenamiento

- ‘learning_rate=1e-4’: Esto indica la tasa de aprendizaje. Para este caso he puesto un valor equilibrado (0.001), para que el modelo no se estanque (valor muy bajo) o no converja (valor muy alto)

- ‘lr_scheduler_type="cosine"’: Este hiperparámetro indica cómo cambia la tasa de aprendizaje según las circunstancias, en este caso, le asignamos el valor ‘cosine’, que quiere decir que cambia de manera suave. De esta forma, podemos obtener un entrenamiento más óptimo

- ‘num_train_epochs=3’: Esto indica el número de dataset, en este caso vamos a poner 3 para evitar overfitting y además porque al entrenar en la nube, el entrenamiento tarda demasiado y los límites gratuitos no me permiten estar más de 4 horas entrenando (que es el tiempo que tarda más o menos en entrenar)

- ‘bf16=True’: En este caso, usa bfloat16, como hemos indicado anteriormente en la cuantización, esto hace que consuma menos VRAM y funcione en GPUs como la T4

- ‘gradient_checkpointing=True’: Esto permite guardar el modelo en cada ‘epoch’, esto lo pongo debido a que tenía esa incertidumbre de que la ejecución se parase por problemas en la nube
- ‘logging_steps=10’: Muestra por consola el progreso por cada diez pasos que entrene. Esto es útil para ver como va progresando el entrenamiento
- ‘save_strategy="epoch"’: Esto indica cada cuanto se debe de hacer el ‘checkpoint’ indicado anteriormente
- ‘eval_strategy="epoch"’: Esto permite que en cada ‘epoch’ haga una evaluación para ver en más detalle el progreso del entrenamiento
- ‘load_best_model_at_end=True’: Como se hacen tres checkpoints totales (3 epoch), al acabar guarda el mejor modelo de los tres checkpoints. Esto puede ayudar de manera que si en el tercer ‘epoch’ el modelo es peor debido a overfitting, guarde el checkpoint anterior
- ‘metric_for_best_model="eval_loss"’: En este hiperparámetro, se indica la métrica que debe de medir para quedarse con el mejor modelo
- ‘optim="paged_adamw_8bit"’: He usado este optimizador ya que se suele usar para las GPUs de las que se dispone en la nube, debido a su bajo consumo de VRAM
- ‘warmup_ratio=0.03’: Este parámetro indica que en los primeros 3% de los pasos totales, el learning rate cambia muy lentamente, de forma que se estabiliza el entrenamiento y no hay cambios bruscos
- report_to="tensorboard": Nos permite ver gráficamente algunos valores durante el entrenamiento gracias a TensorBoard

Por último, nos encontramos los parámetros de la clase ‘SFTTrainer’, que son los siguientes:

- ‘model=model’: Es el modelo al que se le quiere hacer Fine-Tuning
- ‘args=training_args’: Son todos los parámetros explicados anteriormente
- ‘train_dataset=dataset["train"]’: Indica el conjunto de entrenamiento
- ‘eval_dataset=dataset["test"]’: Indica el conjunto de testing

Una vez establecidos los parámetros, es hora de hacer Fine-Tuning. En la siguiente imagen podemos ver diferentes secciones del entrenamiento. En la sección número uno, podemos ver los distintos ‘logging_step’ que se hacen cada 10 pasos, donde se indica el valor de perdida de entrenamiento, el learning rate, entre otros valores. Además, en la sección dos, podemos ver el progreso del entrenamiento. Por último,

en la sección tres podemos ver que se está haciendo la evaluación al terminar un ‘epoch’

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                                    1
{'loss': '0.9876', 'grad_norm': '0.2734', 'learning_rate': '4.636e-05', 'entropy': '0.989', 'num_tokens': '2.192e+05', 'mean_token_accuracy':
'0.757', 'epoch': '1.623'}
{'loss': '1.015', 'grad_norm': '0.3262', 'learning_rate': '4.153e-05', 'entropy': '1.03', 'num_tokens': '2.308e+05', 'mean_token_accuracy': '0
.7563', 'epoch': '1.713'}
{'loss': '1.022', 'grad_norm': '0.3438', 'learning_rate': '3.679e-05', 'entropy': '1.028', 'num_tokens': '2.426e+05', 'mean_token_accuracy': '
0.7553', 'epoch': '1.804'}
{'loss': '1.016', 'grad_norm': '0.4238', 'learning_rate': '3.216e-05', 'entropy': '1.005', 'num_tokens': '2.549e+05', 'mean_token_accuracy': '
0.7548', 'epoch': '1.894'}
{'loss': '0.9744', 'grad_norm': '0.3809', 'learning_rate': '2.771e-05'  2  ntropy': '0.9917', 'num_tokens': '2.67e+05', 'mean_token_accuracy':
'0.7636', 'epoch': '1.984'}
 67%|                                                                    | 222/333 [2:19:34<1:08:08, 36.84s/it]
 12%|                                                                    | 3/25 [00:17<02:11,  5.97s/it]
Ln 78, Col 23   Spaces: 4   UTF-8   LF   Python   3.12.11 ('cloudspace': co                                3
```

Una función interesante del Fine-Tuning, es que gracias a ‘report_to=”tensorboard”’ podemos ver gráficamente cómo va bajando la función de pérdida. Escribimos en una celda anterior al entrenamiento estos comandos:

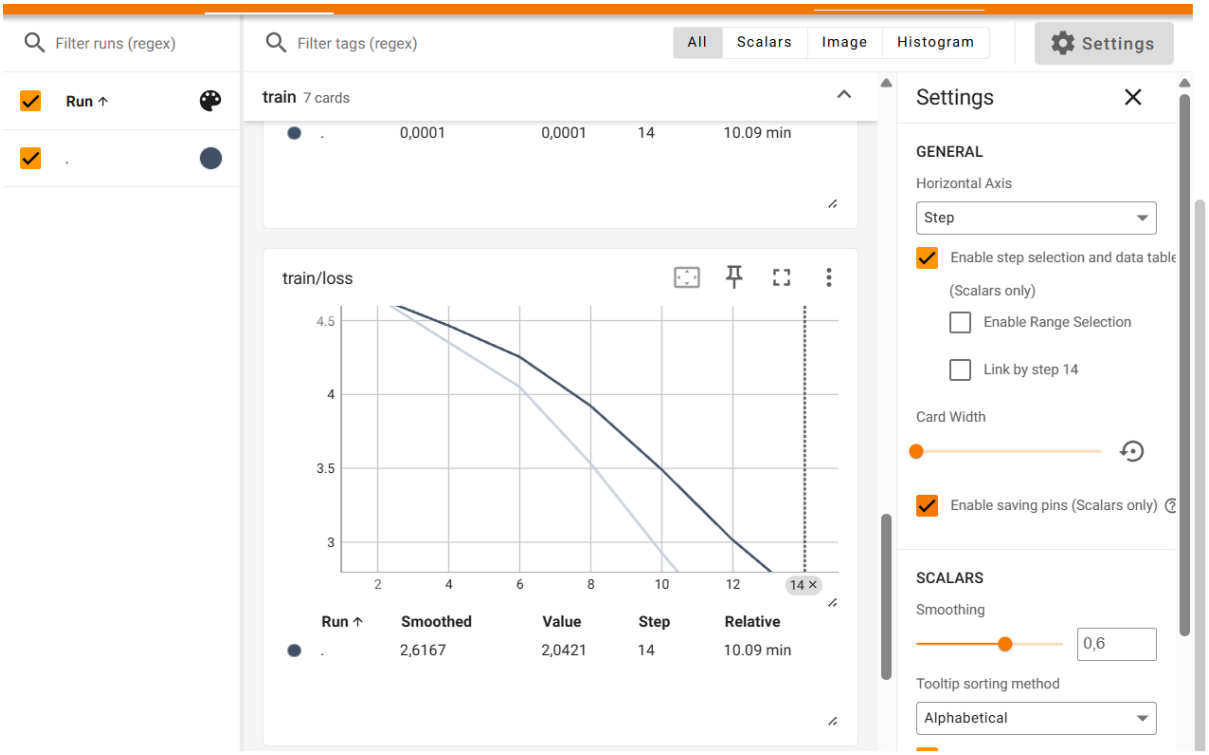
```
%load_ext tensorboard
%tensorboard --logdir ./logs --reload_interval 5
%tensorboard --logdir ./logs
```

Luego, en el código del entrenamiento se establece una variable de entorno que indica donde estará la carpeta con los logs

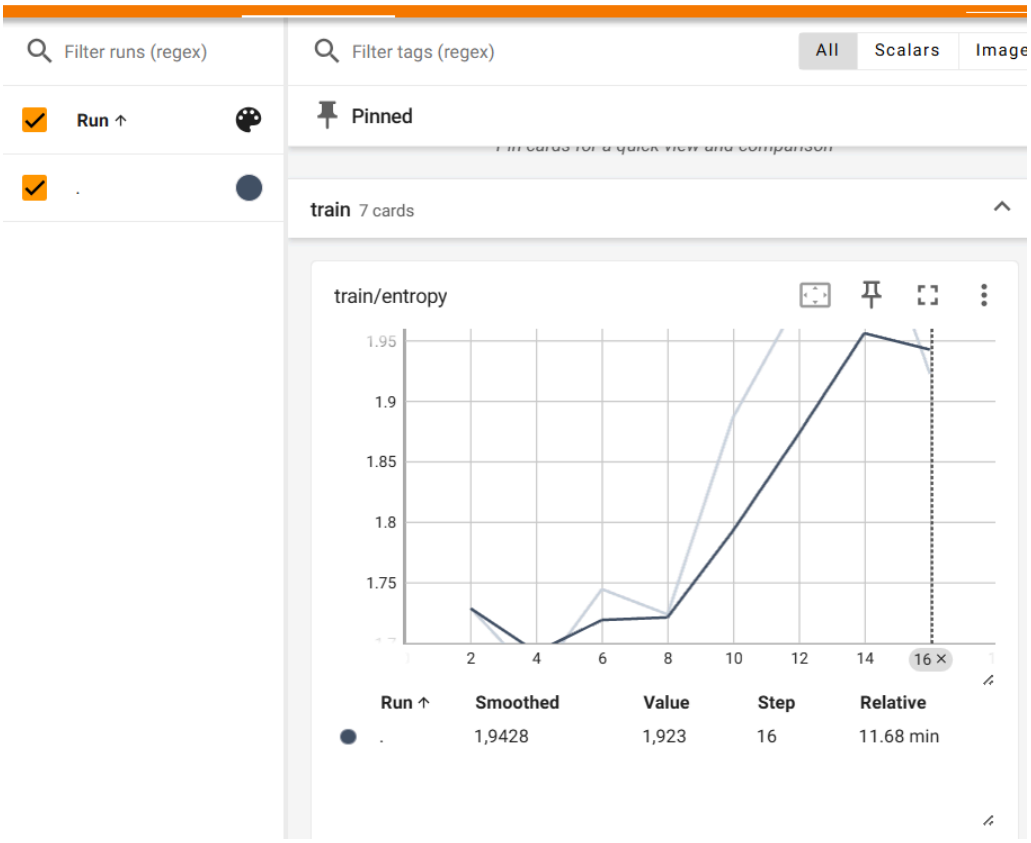
```
import os
os.environ["TENSORBOARD_LOGGING_DIR"] = "./logs"
```

Una vez configurado, se inicia el entrenamiento y podemos ver varias gráficas.

Vemos esta gráfica, donde se muestra el valor de pérdida (eje Y) por cada paso (eje X). Podemos observar que a medida que avanza el entrenamiento, el valor de pérdida va disminuyendo



Otra gráfica interesante es la de “entropy” (explicado posteriormente). Vemos que según avanza el entrenamiento este valor sube y baja, aunque estos valores son de los primeros 16 pasos



Podemos ver que es una herramienta bastante útil para ver el progreso de los distintos valores de interés del entrenamiento

En esta imagen podemos ver el resultado de la evaluación del segundo ‘epoch’, podemos apreciar valores bastante buenos, como que el ‘eval_loss’ es uno (este valor no va de 0 a 1, buscando información, un valor muy alto de este valor es mayor que 4) o el ‘eval_entropy’, que es una medida que cuantifica la incertidumbre en las predicciones del modelo. En este caso, tiene un valor medio (entre 1 y 3 es un valor aceptable). Un valor mayor de 5 quiere decir que el modelo está muy ‘confundido’, haciendo muchas alucinaciones

```
{'eval_loss': '1.011', 'eval_runtime': '174.5', 'eval_samples_per_second': '1.129', 'eval_steps_per_second': '0.143', 'eval_entropy': '1.011',
 'eval_num_tokens': '2.694e+05', 'eval_mean_token_accuracy': '0.7547', 'epoch': '2'}
```

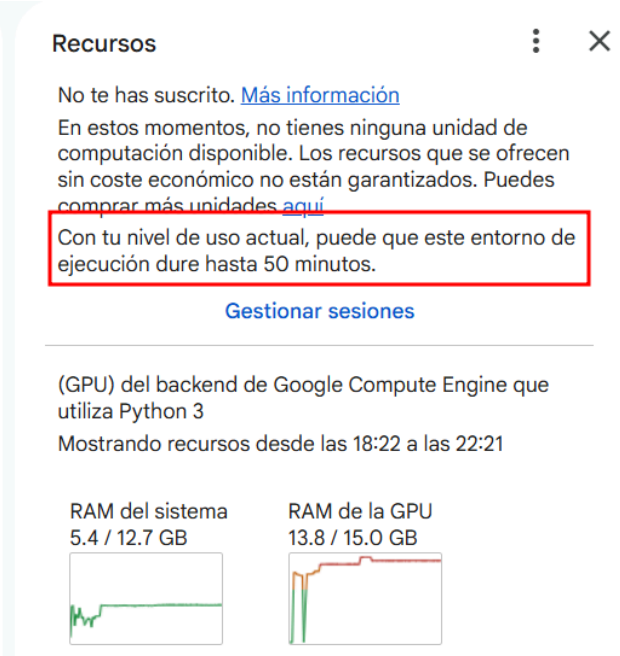
Aquí podemos ver otras imágenes del primer entrenamiento que hice en Google Colab, pero no me sirvió, ya que el entrenamiento iba demasiado lento

```
The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and gen
```

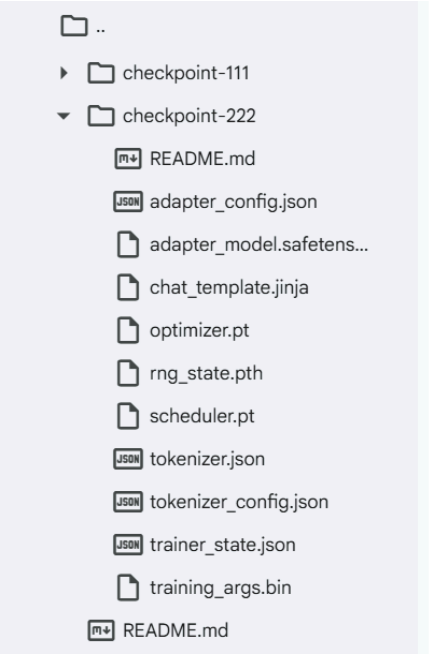
[227/333 3:09:43 < 1:29:23, 0.02 it/s, Epoch 2.04/3]

Epoch	Training Loss	Validation Loss
1	1.144312	1.082687
2	0.910750	0.981144

Si eso le sumas a que el entorno tiene un tiempo límite, en este caso me quedaban 50 minutos de entorno (y al entrenamiento le faltaba una hora y media para terminar), Google Colab no me servía. Lo que hice fue hacerlo en una plataforma llamada ‘LightingAI’ que, sinceramente, deja muchas opciones gratuitas al usuario y me parece, en mi opinión, una muy buena plataforma para hacer entrenamientos de modelos



En esta otra imagen podemos ver que, efectivamente en cada ‘epoch’ se guarda el modelo, con todos los archivos correspondientes



Vemos que ya ha terminado el entrenamiento, y se ha guardado en la carpeta “qwen-finetuned”

```
{'train_runtime': '1.281e+04', 'train_samples_per_second': '0.415', 'train_steps_per_second': '0.026', 'train_loss': '1.151', 'epoch': '3'}
100%|                                                                                                  | 333/333 [3:33:33<00:00, 38.48s/it]
LoRa adapter save to ./qwen-finetuned
```

Una vez entrenado, vamos a unir el adapter creado con el modelo base, de esta forma ya no tendremos que depender del adapter. A este proceso se le llama 'merge'

```python
base_model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-7B-Instruct",torch_dtype=torch.bfloat16,)

tokenizer =  AutoTokenizer.from_pretrained("Qwen/Qwen2.5-7B-Instruct")

#Load and merge

model = PeftModel.from_pretrained(base_model, "3_3_Fine_Tuning/qwen-finetuning")

print("Merged models...")
merged_model = model.merge_and_unload()

#Save the merged model

merged_model.save_pretrained("./qwen-merged")

tokenizer.save_pretrained("./qwen-merged")
```

Evaluación Comparativa y Reflexión

Una vez tengamos el modelo guardado, hacemos las pruebas en un [archivo de python nuevo] con las mismas preguntas de antes (como dice el enunciado) y podemos ver que estas han sido las respuestas del modelo una vez hecho Fine-Tuning:

Pregunta	Respuesta
What does Ellie find out when she returns to her house on The Last of Us?	Ellie finds a note from her father, who is still alive and living in Jackson
Why does Tommy go to Santa Barbara ahead of Ellie in The Last of Us?	Tommy goes to Santa Barbara ahead of Ellie because he is trying to find Abby, who he believes is still alive
What happened in the event "Ellie goes to Seattle" in The Last of Us?	Ellie travels to Seattle, Washington, where she seeks revenge against Abby
Why does Joel kill Marlene in The Last of Us?	Joel kills Marlene because he believes she is trying to kill Ellie
How does the rat king differ from other infected in The Last of Us?	The rat king is a unique and rare type of infected that has been fused together by the Cordyceps fungus, making it stronger and more dangerous than other infected
How do stalkers compare to runners in	Stalkers have a significant strength

terms of strength in The Last of Us?	advantage over runners, being able to easily overpower and kill them
What is the main reason for Ellie's journey to Seattle in The Last of Us?	Ellie travels to Seattle to avenge Joel's death by killing Abby
What happens to Sarah in Texas on The Last of Us?	Sarah is killed by a soldier while trying to escape from the WLF's control
What happens to Sam after he is scratched by an infected in The Last of Us?	Sam is scratched by an infected and turns into a Runner, prompting Joel to kill him
What happened to Ellie and Joel after they left the University on The Last of Us?	Ellie and Joel traveled across the United States, eventually arriving in California
What is the relationship between Jerry and Abby in The Last of Us?	Jerry and Abby are father and daughter, with Jerry being a member of the Washington Liberation Front
How did Abby react to Owen's death in The Last of Us?	Abby was devastated by Owen's death and blamed herself for his death, which led to her emotional breakdown
What motivated Abby to spare Ellie's life in The Last of Us?	Abby spared Ellie's life because she was pregnant and Abby believed that killing Ellie would have been cruel
Why do Clickers develop a form of echolocation in The Last of Us?	Clickers develop a form of echolocation as a result of their blindness and the loss of their eyes. This adaptation allows them to navigate and hunt their prey effectively in dark environments
What are the four categories of collectibles in The Last of Us?	The four categories of collectibles in The Last of Us are Artifacts, Training Manuals, Firefly Pendants, and Comics

En este caso, ha acertado 13 de 15 preguntas, por lo que podemos afirmar que el modelo ha mejorado bastante. Haciendo cálculos con estos resultados (13/15) y los anteriores (5/15), el modelo ha mejorado del 33.3% a un 86.7% después del Fine-Tuning, es decir, ha tenido una mejora porcentual relativa de un 160% ((86.7-33.3)/33.3 * 100), en otras palabras, ahora acierta 2.6 veces más que antes, por lo que el proceso de fine-tuning ha sido claramente efectivo.

No obstante, una de las preguntas que ha acertado puede también darse como no válida, concretamente la pregunta 4 (¿Por qué Joel mata a Marlene?), ya que el modelo dice que Marlene está intentando matar a Ellie y puede darse como cierto

realmente ya que Marlene es la que cuenta la noticia de que van a operar a Ellie del cerebro para la vacuna, lo que conlleva a matarla.

Por otro lado, podemos sacar conclusiones de las preguntas que ha fallado el modelo ajustado. Falla en la primera y en la décima pregunta, lo cual puede deberse a varias causas: falta de ejemplos similares en el dataset de entrenamiento o problemas de generalización.

Además de la mejora cuantitativa, también se observa un cambio cualitativo en el comportamiento del modelo. Las respuestas generadas tras el fine-tuning son más concisas y claras, lo que indica una mejor adaptación a la tarea descrita al principio. Asimismo, se reduce la aparición de alucinaciones, lo que incrementa la fiabilidad general del sistema.

En conclusión, el fine-tuning ha mejorado significativamente el rendimiento del modelo tanto en precisión como en calidad de respuesta, aunque todavía presenta errores en ciertas preguntas, que se podría arreglar simplemente añadiendo más datos en el dataset.

Publicación del modelo

Una vez visto que el modelo funciona bastante bien, vamos a proceder a subir el modelo en Hugging Face, hay varias formas de hacerlo, una de ellas es mediante código, subiendo los archivos que contiene el modelo. El código es el siguiente:

```python
from huggingface_hub import login, upload_folder

# Esto nos permite logearnos en Hugging Face para poder subir el modelo
login()

# Subimos los archivos del modelo
upload_folder(folder_path="./qwen-merged", repo_id="adriangg04/TheLastOfUs-QA", repo_type="model")
```

upload_folder() nos va a permitir subir la carpeta donde se encuentra tanto el modelo como el tokenizador. En el siguiente enlace podemos ver que el modelo se ha subido correctamente, con su README correspondiente

Publicación de espacios

Por último, vamos a publicar dos espacios acerca del modelo. Uno de ellos es un espacio donde aparezcan los dos modelos a la vez, la desventaja de esto es que al estar los dos modelos, generar ambas respuestas es demasiado lento.

Luego, en el segundo space puedes elegir un modelo u otro, de esta forma, se genera la respuesta bastante más rápido

Enlaces a los repositorios de Hugging Face

Dataset generado para entrenar el modelo:
https://huggingface.co/datasets/adriangg04/the-last-of-us-instruction-dataset

Modelo ajustado para preguntas y respuestas de Hugging Face:
https://huggingface.co/adriangg04/TheLastOfUs-QA

Space con los dos modelos a comparar (generar una respuesta puede tardar hasta 10 minutos): https://huggingface.co/spaces/adriangg04/TheLastofUs-QA

Space donde puedes elegir un modelo de los dos (se genera respuestas mucho más rápido):
https://huggingface.co/spaces/adriangg04/Qwen-Finetuned-TheLastofUs

ANEXO

A la hora de establecer los hiperparámetros del Fine-Tuning, se ha hecho otra prueba para evaluar el conjunto de testing con otra métrica, en este caso una métrica llamada BERTScore y ROUGUE que compara el significado y similitud del conjunto de testing con el resultado del modelo. Este sería la función para hacer uso de dichas métricas:

```python
def compute_metrics(eval_preds):
    predictions, labels = eval_preds

    # En modelos causales, puede venir como tupla
    if isinstance(predictions, tuple):
        predictions = predictions[0]

    # Convertimos logits a ids si es necesario
    if predictions.ndim == 3:
        predictions = predictions.argmax(-1)

    # Reemplazar -100 en labels (ignorar padding)
    labels = [[(l if l != -100 else tokenizer.pad_token_id) for l in label] for label in labels]

    # Decodificar
    decoded_preds = tokenizer.batch_decode(predictions, skip_special_tokens=True)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

    # ROUGE
    rouge_result = rouge.compute(
        predictions=decoded_preds,
        references=decoded_labels,
        use_stemmer=True
    )

    # BERTScore
    bertscore_result = bertscore.compute(
        predictions=decoded_preds,
        references=decoded_labels,
        lang="es"
    )

    return {
        "rougeL": rouge_result["rougeL"],
        "bertscore_f1": sum(bertscore_result["f1"]) / len(bertscore_result["f1"]),
    }

# Se define la clase para hacer Fine-Tuning
trainer = SFTTrainer(
    model=model, # Indicamos el modelo
    args=training_args, # Indicamos todos los argumentos anteriores
    train_dataset=train_dataset, # Se indica el dataset de entrenamiento
    eval_dataset=eval_dataset, # Se indica el dataset de evaluación
    compute_metrics=compute_metrics
)
```

Luego, en la clase TrainingArguments tenemos que poner los distintos parámetros:

- ➢ metric_for_best_model="eval_bertscore_f1"
- ➢ greater_is_better=True

Esto hace que la evaluación del modelo se haga por las métricas indicada (‘eval_bertscore_f1’) y se quede con la que sea mayor.

Pero esta prueba me dió problemas, porque al parecer necesita mucha más memoria RAM, por lo que me da el siguiente error:

```
# Se define la clase para hacer Fine-Tuning
trainer = SFTTrainer(
    model=model, # Indicamos el modelo
    args=training_args, # Indicamos todos los argumentos anteriores
    train_dataset=train_dataset, # Se indica el dataset de entrenamiento
    eval_dataset=eval_dataset, # Se indica el dataset de evaluación
    compute_metrics=compute_metrics
)

trainer.train() # Se empieza el entrenamiento
trainer.save_model(OUTPUT_DIR) # Una vez entrenado, se guarda el mdoelo en el directorio
tokenizer.save_pretrained(OUTPUT_DIR) # Se guarda también el tokenizador
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
    return torch_pad_and_concatenate(tensors, new_tensors, padding_index=padding_index)
           ^^^
```